

Discovering Biases in Information Retrieval Models Using Relevance Thesaurus as Global Explanation

Anonymous ACL submission

Abstract

Most efforts in interpreting neural relevance models have focused on local explanations, which explain the relevance of a document to a query but are ineffective in predicting the model’s behavior on unseen texts. We propose a novel method to globally explain neural relevance models by constructing a “relevance thesaurus” containing semantically relevant query and document term pairs. This thesaurus augments BM25 scoring functions to better approximate the neural model’s predictions. Our method involves training a neural relevance model to score the relevance of partial query and document segments, which is then used to identify relevant terms across the vocabulary space. We evaluate the resulting thesaurus explanation based on ranking effectiveness and fidelity to the targeted neural ranking model. Notably, our thesaurus reveals the existence of brand name bias in ranking models, demonstrating the utility of our explanation method.

1 Introduction

Transformer-based information retrieval (IR) models (Dai and Callan, 2019; MacAvaney et al., 2019) that are trained on large datasets like MS MARCO (Nguyen et al., 2016) are very effective in predicting relevance between a query and document. Contextual representations in these models enable semantic matching, such as matching the query term “car” with the document term “vehicle”. However, it is challenging for researchers to predict the potential failures of a model, such as when it matches a query term to non-relevant document terms.

Another potential risk associated with neural retrieval models is an unintended bias toward certain entities or groups (May et al., 2019). While it is appropriate for a model to associate the query term “car” with various car brand names (e.g., Ford), the model should not exhibit a strong preference

Query term	Document term			
injury	injure 0.26	wound 0.24	torn 0.19	...
when	24th 0.33	2010 0.11	2015 0.01	...
car	vehicle 0.68	ford 0.38	honda 0.28	...
cud	cudâ 0.50	cuda 0.50	...	...

Table 1: Example entries from our relevance thesaurus. Unexpected behaviors found by our method are highlighted.

for a particular brand, leading to the model favoring that brand over another when all other factors are identical. For the safe deployment of information retrieval (IR) models in real-world scenarios, more detailed explanations of model behavior are required, such as providing which lexical expressions are considered relevant by the models.

To address these challenges and mitigate potential risks, post hoc explanation methods for black box machine learning models can be employed. Most existing IR model explanations are local explanations, which focus on individual model predictions, such as a specific query-document pair (Kim et al., 2022) or a ranked list for a query (Verma and Ganguly, 2019; Llordes et al., 2023). These explanations indicate certain document terms are responsible for relevance to a specific query. However, local explanations have two major limitations that hinder their ability to infer cases where the model will exhibit unexpected behaviors.

First, the explanations are limited to the terms observed in the given query and document, and biases might be present in queries or documents that were not evaluated or inspected with the explanations. Second, the attributions to document terms might be highly dependent on the given contexts

and may not generalize to different contexts.

To overcome these limitations, we propose building a global explanation (Guidotti et al., 2018) that provides lexical insights. We can describe a model’s behavior in a compact and interpretable structure that is not limited to a specific instance.

Our proposed global explanation focuses on identifying relevant pairs of query and document terms that can effectively explain the matching behavior of neural retrieval models. We refer to this format of explanation as a *relevance thesaurus*, with examples illustrated in Table 1. The table indicates that, if a query contains term “injury,” then it is likely to be matched with document terms “injure,” or “wound,” with the former being the more likely. This allows researchers to anticipate which terms, when contained in a document, would make the model predict higher relevance for that document, without the need to inspect the actual document itself.

Constructing a relevance thesaurus is challenging due to the large number of potential term pairs. While many local (Ribeiro et al., 2016) and global explanation (Han et al., 2020) methods build a candidate set of features from data and adjust scores based on the target model’s outputs, this strategy becomes infeasible when the number of features increases to billions, as in our case. To overcome this challenge, we propose a novel approach that trains an intermediate neural model, PaRM (Partial Relevance Model), from the target model and uses it to infer important features.

PaRM is designed to predict a score for a term pair, which is then used to predict the score for the query-document pair to which the term pair belongs. By training PaRM with knowledge distillation from the target neural model to be explained, we ensure that the generated relevance thesaurus faithfully explains the target model’s behavior.

Rather than assessing the accuracy of each term pair in the relevance thesaurus individually, the thesaurus is extrinsically evaluated by integrating it into lexical matching models (BM25 (Robertson et al., 2009) and QL (Ponte and Croft, 1998)), adding interpretable semantic matching to them. The resulting retrieval methods are evaluated based on retrieval effectiveness and fidelity to the target neural retrieval models. The results on multiple datasets show the effectiveness of the acquired relevance thesaurus.

To demonstrate the use case of our relevance

thesaurus, we introduce three unexpected findings about the behavior of neural retrieval models trained on MS MARCO, which were revealed from the analysis of the relevance thesaurus: (1) the *car-brand bias*, which suggests that models exhibit biases towards certain car brands; (2) the *temporal bias*, which indicates that models consider distant future or past years to be more strongly associated with the query term “when” compared to the current year; (3) the *postfix-a* finding, which reveals that models treat the character “a” appended to a term as equivalent to a quotation mark due to encoding errors.

Experiments using multiple state-of-the-art neural information retrieval models demonstrate that these behaviors are not limited to the cross-encoder ranker which is used to distill the relevance thesaurus but are also the case in multiple other IR models, Splade (Formal et al., 2021b) and Contriever (Izacard et al., 2021). This highlights the importance of global explanations for retrieval models.

2 Related works

2.1 Global Model Explanations

Large portions of works on global explanations are for classification tasks on tabular features (Craven and Shavlik, 1995; Boz, 2002; Guidotti et al., 2018). They cannot be applied to the Transformer model for token sequences, as tabular features are not defined. Instead, global explanation works in the NLP domain target single text classification, by attributing output labels to some words or phrases (Rajagopal et al., 2021; Han et al., 2020). However, these word-to-output label attributions are not applicable for explaining text pair models like IR models, because document terms’ importance is highly dependent on queries. It would make a more meaningful explanation if it indicate certain terms or phrases from the query are associated with specific terms or phrases that appear in the document.

2.2 Explanations for Neural IR

Existing neural IR models (Khattab and Zaharia, 2020; Gao et al., 2021; Formal et al., 2021b; Nogueira et al., 2019) encode entire queries and/or documents with a single Transformer network and are not capable of encoding parts of the query/document in the absence of the remaining context. Applying perturbation-based explanation

approaches (Kim et al., 2020; Ribeiro et al., 2016) on these models can be problematic because removing tokens from a query could lead to a larger change of the meaning in IR tasks than the other NLP tasks. For example, a document that is relevant to the query “ACL location” is not relevant to the query “location”, as the relevant document for this query needs to describe what the expression “locations” means rather than mentioning any location.

Existing IR models explanations (Verma and Ganguly, 2019; Llordes et al., 2023; Lyu and Anand, 2023; Pandian et al., 2024; Naseri et al., 2021) mostly work in the query-level, and output terms for one query cannot be used to infer the model’s behaviors in other queries. Chowdhury et al. (2023) target explaining categorical features in learning-to-rank IR models, while we target sequence processing Transformer models.

To enhance lexical models with recent advances, Boytsov and Kolter (2021) proposed fine-tuning BERT (Devlin et al., 2019) for the translation language model (Berger and Lafferty, 1999). This approach, however, is limited to the semantic matches between terms in BERT’s subword vocabulary and does not extend to terms formed from multiple subwords. Moreover, the work lacks analysis or evaluation regarding the explanation perspectives and does not provide qualitative insights from the outcomes.

3 Relevance Thesaurus Building

We define the global model explanation problem for an information retrieval model, based on the definition by Guidotti et al. (2018), as follows:

Model Explanation Problem. Given a blackbox relevance predictor S_b that takes a query q and a document d as inputs and predicts a relevance score $y \in Y$, the global model explanation problem aims to find a human-interpretable explanation E and an explanation logic ϵ . The explanation logic ϵ is a function that converts the explanation E into a global predictor S_e . The global predictor S_e predicts a score $\hat{y}$ for (q, d) , which approximates the prediction prediction y .

As a blackbox predictor S_b to be explained, we target the full cross-encoder (CE) document ranking model (Dai and Callan, 2019), which takes the concatenated sequence $q; d$ into the Transformer encoder to predict the relevance. As a format for an explanation E , we use a relevance thesaurus,

which is a set of triplets (qt, dt, s) , where qt is a query term, dt is a document term, and s is the score assigned to the term pair.

To build an interpretable predictor S_e , we augment the relevance thesaurus (E) into the BM25 scoring function to handle vocabulary mismatches between queries and documents. The relevance thesaurus captures semantic relationships between terms that may not be explicitly present in the query or document. This will work to fill the gap between BM25 and CE, providing a more interpretable explanation of the CE model’s behavior and improving the retrieval performance of the sparse retrieval model BM25.

Many local and global explanation methods explicitly build a candidate set of features (e.g., terms) from data. To determine the appropriateness of each candidate feature as an explanation, these methods initially assign scores to the features. Then, the scores are adjusted based on the observed model behaviors. This strategy maintains explicit feature candidates and their scores throughout the optimization process, which can be challenging if the number of features increases, as in our explanation format, where the term pairs can scale to billions.

To handle this challenge, we propose to implicitly optimize them using an intermediate neural model and generate per-feature scores from the intermediate model. Our intermediate model predicts relevance scores for term pairs, and term-level scores (E) are used to globally approximate (S_b) the blackbox model (S_b)’s predictions for query-document pairs.

As an intermediate neural model, we propose PaRM (Partial Relevance Model), which is designed to score relevance between partial segments of query and document. Unlike the original cross-encoder models or other relevance models that assign a score for an entire query and document, PaRM can predict meaningful scores for partial queries or documents

This is important because the original CE models cannot accurately assess the contributions of individual tokens when they are isolated from their original contexts. For example, if the query is “Who is Plato” and the document is a single term “Plato”, the original CE models will likely predict relevance scores indicating non-relevance, as a document with a single term is unlikely to provide meaningful information to users. PaRM, on the other hand, predicts a score that indicates par-

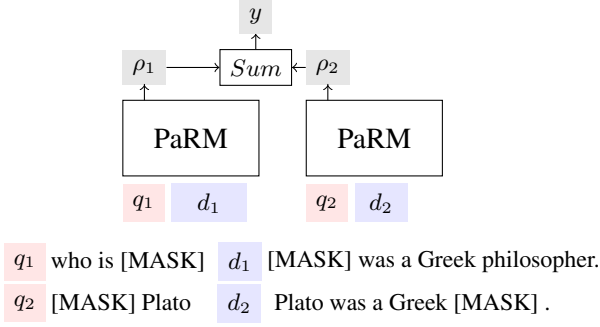


Figure 1: The first phase of training PaRM. The query “who is Plato” (red) is partitioned into q_1 and q_2 . The document “Plato was a Greek philosopher” (blue) is masked to generate d_1 and d_2 .

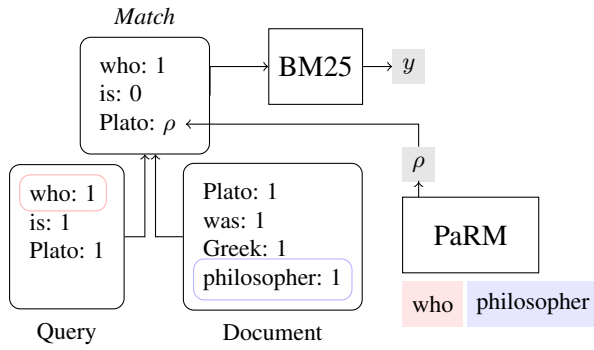


Figure 2: The second phase of training PaRM. BM25 computes a relevance score based on each query terms frequency in the document (*Match*). If some query term (e.g., who) does not appear in the document, the PaRM score ρ for the most relevant document term is used as a discounted query term frequency.

tial relevance, which can be combined with partial relevance scores for other terms to build the final relevance score for the query-document.

PaRM is trained end-to-end by distilling predictions from the CE model. However, a challenge arises because PaRM is expected to predict a score for a query term and document term, while the available signal is only at the query-document level. If we can infer alignments that indicate which term-pairs are responsible for the query-document relevance, we can supervise the term-pair level scores using the CE model. The novelty of PaRM training lies in the ability to infer alignments with no given alignments.

In the first stage, we train PaRM to predict scores for two partial query-document segment pairs, which only require a crude level of alignment, such as attention scores. After the first stage, we use the PaRM model to infer word-level alignments. In the second stage, we use the inferred

alignments to further fine-tune the PaRM model to predict a well-scaled score for word-level relevance, which is used to form the final explanations (E).

3.1 PaRM first phase training

The first phase PaRM predicts the relevance score S_e for a given query-document pair by generating scores for two partial inputs, (q_1, d_1) and (q_2, d_2) . These inputs are built by extracting a continuous span from the query q to form q_1 and using the remaining tokens with a [MASK] token for q_2 .

The corresponding document segments, d_1 and d_2 , are constructed by masking tokens from the document d that have low attention scores for q_1 and q_2 , as determined by the full cross-encoder (CE) ranker.

We randomly select how many tokens to be left in d_i , ranging from one to all tokens of d (see Appendix D for details). In most cases, d_i contains sufficient evidence to learn relevance while still allowing for a few extreme cases where either only a single query term or a single document term is present.

Scores for (q_1, d_1) and (q_2, d_2) are obtained by projecting BERT’s CLS pooling representations.

$$\text{PaRM}(q_i, d_i) = W \cdot \text{BERT}_{CLS}(q_i; d_i) + b \quad (1)$$

The final score for the query and document pair is the sum of the scores from two partial views.

$$S_e(q, d) = \text{PaRM}(q_1, d_1) + \text{PaRM}(q_2, d_2) \quad (2)$$

The combined score S_e is trained from the scores (S_b) of the targeted black box model (CE) using margin mean square error (MSE) loss (Hofstätter et al., 2021) on relevant and non-relevant query-document pairs.

$$\mathcal{L} = \text{MSE}(S_e(q, d^+) - S_e(q, d^-), S_b(q, d^+) - S_b(q, d^-)) \quad (3)$$

Once PaRM is trained, we can use it to score an arbitrary query span or document span, including a single term. However, the scores are only trained for ranking and not calibrated to a specific range, which makes it hard to determine which term-pair entries have a sufficiently large score to be included in the relevance thesaurus.

3.2 Fine-tuning for Term Matching

In the second phase, we fine-tune PaRM so that it scores the relevance of a query term qt and a document term dt on a scale from 0 to 1. Specifically, we consider the scenario of augmenting BM25 by handling vocabulary mismatch based on the scores from PaRM.

We consider a query-document pair that any query term is missing in the document. We assume that the document term that has the highest PaRM score against the corresponding query term is most likely to be relevant to the query term if any term is relevant. We then use the output of PaRM to replace the term frequency (Figure 2). If the assumed pair is relevant, it will be more likely to appear in the relevant document and will be trained to score higher, and non-relevant ones will appear in the non-relevant document and be trained lower.

For a pair of query q and document d , if any query term does not have an exact match in the document, we randomly select one query term qt to be trained. All document terms are scored against qt using $\text{PaRM}(qt, dt)$ and the document term dt with the highest score is paired with qt . Note that terms are not from the BERT tokenizer, but are from the tokenizer developed for BM25. Thus, a single term can contain multiple BERT subwords.

The training network is defined as follows. To ensure the output is between 0 to 1, we apply a sigmoid layer (σ) on top of the projected output.

$$\text{PaRM}(qt, dt) = \sigma(W \cdot \text{BERT}_{\text{CLS}}(qt; dt) + b) \quad (4)$$

In the original BM25 formula, the score for the query term qt is determined by qt 's document frequency, $tf_{qt,d}$. We modify BM25 so that, for a query term that does not appear in the document, we replace $tf_{qt,d}$ with the output of $\text{PaRM}(qt, dt)$.

$$f(qt, d) = \begin{cases} tf_{qt,d} & \text{if } qt \in d \\ \text{PaRM}(qt, dt) & \text{if } qt \notin d \end{cases} \quad (5)$$

Note that $tf_{qt,d}$ can be large but PaRM is bounded above by 1, so an implied match is never stronger than a term that matches exactly at least once. The relevance score is computed based on the BM25 scoring function:

$$S_e(q, d) = \sum_{qt \in d} \text{IDF}(qt) \cdot \frac{f(qt, d) \cdot (k_1 + 1)}{f(qt, d) + K} \quad (6)$$

where K is a function of document lengths, which is independent of $f(qt, d)$.

PaRM is trained end-to-end from the pairwise hinge loss between a relevant pair (q, d^+) , and a non-relevant pair (q, d^-) :

$$\mathcal{L} = \max(0, 1 - S_e(q, d^+) + S_e(q, d^-)) \quad (7)$$

Note that we do not use knowledge distillation here, because the output scores scale of the BM25 scoring function is not easily adjustable and may not be possible to match the score margin of the neural ranking model.

During the training phase, the equations from 4 to 7 are implemented inside a neural network framework, and the gradient to the loss $\mathcal{L}$ is back-propagated to train PaRM's parameters. Note that PaRM scores for selecting the highest scored dt are pre-computed with the model after the first phase.

After PaRM is fine-tuned, it can pre-compute the scores for potential qt and dt candidates. These candidate pairs and scores compose a relevance thesaurus. The acquired relevance thesaurus can be used to either inspect the model's behavior or used with the BM25 scoring function.

We name the modified BM25 function which handles non-exact lexical matches based on the relevance thesaurus as BM25T (BM25 with The-saurus). For each query term qt , if qt is found in the document d , relevance thesaurus is not used. If qt is not found, the document term $dt \in d$ with the highest (pre-computed) $\text{PaRM}(qt, dt)$ score in the thesaurus is used to compute the score.

4 Experiments

4.1 Implementation

As a target ranker to be explained, we use a publicly available cross-encoder, which is fine-tuned from distilled-BERT¹. The predictions of this model are used as teacher scores in Equation 3. We initialized PaRM with pre-trained BERT-based-uncased. The maximum sequence length of the input in the first and second phases of training PaRM is set to 256 and 16 tokens, respectively.

The models are trained on the widely used MS MARCO passage ranking dataset (Nguyen

¹cross-encoder/ms-marco-MiniLM-L-6-v2 from <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-12-v2>

et al., 2016). BM25 and BM25T use the tokenizer from the Lucene library ² with the Krovetz stemmer (Krovetz, 1993), preferred over the Porter stemmer (Porter, 1980) for producing actual words.

Relevance Thesaurus Building PaRM scores the candidate term pairs to build the final relevance thesaurus as a global explanation of the full cross-attention ranking model. The candidates are drawn from the frequent terms in the MS MARCO corpus. The top 10K frequent terms were considered as query terms, and the top 100K terms as document terms, resulting in 10^9 pairs. Due to the short input length, the computing time for each input is significantly faster than that for longer sequences in the full cross-attention ranker. Only candidate term pairs scoring above 0.1 are included in the relevance thesaurus, resulting in a total of 553,864 term pairs.

4.2 Evaluations

We evaluate the BM25T model by exploiting our built relevance thesaurus in two ways: ranking effectiveness and fidelity. Ranking effectiveness is measured by standard ranking evaluation metrics evaluating the ranked lists based on ground truth judgments. It demonstrates to what extent BM25T can be used for relevance ranking. Fidelity expresses the extent to which the BM25T faithfully explains the behavior of the target ranking model, i.e., the cross-encoder model.

To demonstrate the generalizability of the relevance thesaurus obtained from PaRM, we developed QLT (Query Likelihood with Thesaurus), a variant of BM25T (PaRM) based on the query-likelihood (QL) framework (Ponte and Croft, 1998). QLT incorporates the translation language model (Berger and Lafferty, 1999) and uses translation probabilities extracted from our relevance thesaurus. Unlike BM25T which computes the query-term’s score based on one most relevant document term, QLT computes the score as the sum of the relevance score of the document terms.

To provide a baseline comparison and demonstrate the effectiveness of PaRM in building a relevance thesaurus, we re-purposed a local explanation method (Llordes et al., 2023) as a global explanation (Lundberg et al., 2020), denoted as BM25T (L to G). The original method identifies query-matching document terms for a given query

Model	TREC DL19 NDCG@10	TREC DL20 NDCG@10	Dev MRR
BM25	0.516	0.503	0.160
BM25T (L to G)	0.518	0.501	0.158
BM25T (PaRM)	0.550 [‡]	0.546 [‡]	0.180 [‡]
QL	0.495	0.509	0.153
QLT (PaRM)	0.543 [‡]	0.540 [‡]	0.170 [‡]
Cross-encoder	0.763	0.739	0.375

Table 2: Ranking performance on the MS MARCO driven datasets. [‡] marks the statistically significant difference ($p < 0.01$) to the baseline in each group.

and its ranked documents. We adapted it by aligning each document term to the most relevant query term using a cross-encoder and aggregating alignments across 400K training queries. The aggregated term pair scores form a relevance thesaurus, which augments the BM25 scoring function as in BM25T (PaRM) (See Appendix B for details).

In-domain ranking effectiveness. First, we evaluate ranking effectiveness on three datasets derived from MS MARCO. TREC DL 2019 and 2020 (Craswell et al., 2020, 2021) contain 43 and 53 queries, respectively. Top-ranked documents are thoroughly judged by NIST assessors, which makes them more reliable for evaluating the ranking effectiveness.

We also used a larger dataset called MS MARCO-dev, which we built by sampling 1,000 queries from the development split of MS MARCO. As this dataset is sparsely judged, with most queries having only one relevant document, we evaluated it using mean reciprocal rank (MRR). MS MARCO-dev will also be used for evaluating fidelity, where more data points are preferable.

Table 2 shows the ranking effectiveness of methods on the MS MARCO datasets. BM25T with our proposed PaRM shows significant gains ($p < 0.01$) over BM25 in all datasets. The obtained gains demonstrate that the distilled relevance thesaurus effectively improves the vocabulary mismatch problem of BM25. In contrast, the BM25T (L to G) does not show consistent improvements. BM25T still has a gap from the cross-encoder model, showing room for improvement in future work. Note that we do not include other retrieval models to compare with their ranking effectiveness, as they cannot be used to make a global explanation. QLT (PaRM) also has better effectiveness compared to QL. Considering that the thesaurus is only tuned for BM25 but not for QL,

²<https://lucene.apache.org/>

Dataset	BM25	BM25T	Cross Encoder
HotpotQA	0.633	0.641 [†]	0.725
DBPedia	0.325	0.350 [†]	0.447
NQ	0.307	0.332 [†]	0.462
Touché-2020	0.499 [†]	0.337	0.272
SCIDOCS	0.150	0.148	0.163
TREC-COVID	0.583	0.602	0.733
FiQA-2018	0.245	0.248	0.341
Quora	0.775 [†]	0.738	0.823
ArguAna	0.407 [†]	0.359	0.311
SciFact	0.678	0.678	0.688
NFCorpus	0.319	0.348 [†]	0.369
ViHealthQA	0.217 [†]	0.173	0.168

Table 3: The ranking effectiveness measure (NDCG@10) of the methods on BEIR datasets. [†] marks the statistically significant difference ($p < 0.05$) between BM25 and BM25T

this result demonstrates the generalization ability of the acquired relevance thesaurus.

Out-of-domain ranking effectiveness. The BEIR benchmark (Thakur et al., 2021) is a collection of IR datasets and is widely used to measure the generalizability of models without domain-specific training. We evaluate the zero-shot ranking effectiveness of the BM25T model over this benchmark, using the same relevance thesaurus distilled from the cross-encoder model that is trained on MS MARCO.

Table 3 shows evaluation results on the BEIR datasets. Out of the 12 datasets, the performance difference between BM25 and BM25T is statistically significant ($p < 0.05$) in 8 datasets. Among these datasets, BM25T outperforms BM25 on 7 datasets, showing a performance closer to that of the cross-encoder. Thus, we conclude that the relevance thesaurus is not limited to the corpus on which it is trained and can effectively perform semantic matching in out-of-domain datasets.

Fidelity. As our task is a ranking task, we measure faithfulness in terms of the correlation between the scores from the explanations and the target model.

Given a query q and its corresponding candidate documents $\{d_1, d_2, \dots, d_n\}$, the fidelity of an explanation is computed as the correlation between the scores $\{S_b(q, d_1), S_b(q, d_2), \dots, S_b(q, d_n)\}$ from the targeted neural model and the global predictor from explanations (e.g., BM25T) $\{S_e(q, d_1), S_e(q, d_2), \dots, S_e(q, d_n)\}$.

For the fidelity score for a dataset, we calculate this correlation for each query and then av-

erage them across all queries in the dataset. For each query, the top 1,000 documents retrieved by BM25 are used as candidate documents. The Pearson correlation coefficient, which ranges from -1 to 1, is used as a measure of correlation, with 1 indicating the strongest positive correlation. Appendix C reports results for other correlation measures, which show similar findings.

In addition to the cross-encoder model, which was used for training PaRM, we apply fidelity evaluation to the other IR models that fine-tune Transformer-based models on MS MARCO. Four popular retrieval models are included. The first two models are TAS-B (Hofstätter et al., 2021) and Splade v2 (Formal et al., 2021a) which are trained using knowledge distillation from the cross-encoders. The next two are Contriever and Contriever+M (Izacard et al., 2021). Contriever is trained with unsupervised learning unsupervised, and Contriever+M is the model that further fine-tunes Contriever using MS MARCO. These four models are dual-encoders, where the query and document are independently encoded into vectors using Transformer encoders.

Ranking Model	Fidelity		Ranking
	BM25	BM25T	MRR
Cross-encoder	0.484	0.580	0.375
Splade v2	0.490	0.583	0.335
TAS-B	0.421	0.513	0.318
Contriever	0.417	0.454	0.174
Contriever+M	0.411	0.495	0.307

Table 4: Fidelity of the explanations to the ranking models, measured by Pearson correlations on the MS MARCO Dev dataset. Both BM25 and BM25T are considered explanations for the corresponding ranking models. The ranking performance, measured by Mean Reciprocal Rank (MRR), is provided as a reference.

Table 4 shows the fidelity of BM25 and BM25T to these neural retrieval models on MS MARCO-dev. First, we can observe that in all ranking models, BM25T has higher fidelity than BM25. Also, the gain is larger on models that are trained on MS MARCO. Contriever is not trained based on the MS MARCO dataset, on which BM25T showed the lowest fidelity and smallest fidelity gain.

We conclude that the relevance thesaurus can serve to explain the behavior of the models that are trained with similar training data.

The fidelity evaluations on BEIR datasets also confirm that BM25T is more faithful than BM25 in explaining the cross-encoder model (Table 10).

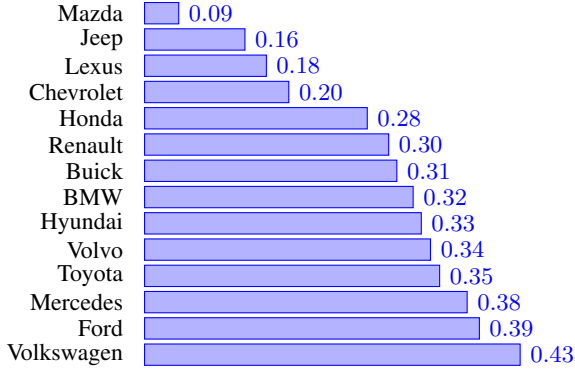


Figure 3: Scores for car brand names against the query term “car” based on our relevance thesaurus.

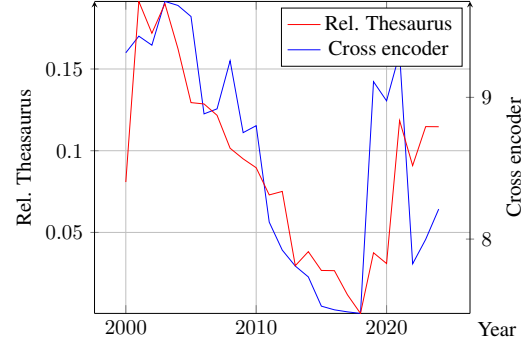


Figure 4: Relevance scores for the query term “when” and document terms representing years from 2000 to 2024, based on a relevance thesaurus and a cross-encoder model.

The fidelity of BM25T is higher than BM25 across all datasets, except in the Quora dataset. The average fidelity across the datasets improved from 0.507 with BM25 to 0.630 with BM25T.

4.3 Insights from the Relevance Thesaurus

The relevance thesaurus contains both reasonable and unexpected term pairs, as illustrated in Table 1. Through the analysis of the thesaurus, we identified three interesting findings.

Car-brand bias The thesaurus reveals that the models associate “car” with many brand names, but assign higher scores to certain brands over others (Figure 3). For example, the pair (“car”, “Ford”) has a score of 0.39, while (“car”, “Honda”) has a score of 0.28.

This bias can impact the ranking of documents in the following way: imagine a query containing the term “car” and two documents that are identical except for the mentioned car brand - one document includes a high-scoring brand, while the other features a low-scoring brand. Due to the higher score assigned by the thesaurus, BM25T will rank the document with the high-scoring brand above the one with the low-scoring brand. This observation suggests that the neural ranking model is likely to exhibit the same bias, prioritizing documents that mention high-scoring brands over those with low-scoring brands, even when the documents’ content is otherwise the same.

When-year bias The models exhibit a temporal bias, assigning different scores to various years for the query term “when”, with much lower scores for years around 2015, when the MS MARCO dataset was constructed (Figure 4). We hypothesize this bias exists because the current year is

often less informative for “when” questions, as more specific temporal information is typically expected. While effective for 2015 data, it could lead to sub-optimal performance for different current years, such as 2024.

Experiments detailed in Appendix A validate the presence of these behaviors in state-of-the-art relevance models, supporting that the behaviors specified by the relevance thesaurus are well representing the neural ranking models.

Postfix a Many thesaurus entries consist of cases where the document term is the query term with an additional “a” or “â” at the end, such as the (“car”, “vehicleâ”). This is due to encoding errors in the MS MARCO dataset, where the right quotation marks (’) were incorrectly decoded as “â”. Combined with the BERT tokenizer normalizing “â” to “a”, this could result in incorrect matches, e.g., considering “cud” (food in cows’ stomach) relevant to “CUDA” (parallel computing platform).

5 Conclusion

We explored using a relevance thesaurus as a global explanation for neural ranking models. We proposed an effective approach for constructing the thesaurus by training a partial relevance model (PaRM). Augmenting the acquired thesaurus into BM25 enhanced its ranking effectiveness and fidelity to the targeted neural ranking model across multiple information retrieval datasets. Furthermore, the thesaurus uncovered unexpected corpus-specific behaviors and biases of state-of-the-art ranking models, highlighting its value in identifying potential issues and limitations in neural rankers.

655 Limitations

656 Our explanation methods have shown the ex-
 657 istence of bias in models trained on the MS
 658 MARCO dataset. However, it is important to note
 659 that the presence of bias does not necessarily indi-
 660 cate inappropriate behavior. In some cases, bias
 661 may actually contribute to effective ranking, as
 662 certain keywords have a higher likelihood of being
 663 relevant due to their ability to be used in different
 664 contexts, such as referring to a person or other en-
 665 tities.

666 Our work has the risk to create unwarranted
 667 distrust in neural ranking models. The presence
 668 of corpus-specific features, such as encoding ar-
 669 tifacts, does not inherently signify a weakness in
 670 the model, as deployed models may be capable of
 671 handling documents with similar encoding errors.

672 While our term replacement experiments were
 673 designed to control for context by maintaining
 674 identical conditions, real-world documents often
 675 exhibit diverse contexts that could potentially di-
 676 minish the impact of biases. In practice, the con-
 677 textual differences between documents may result
 678 in greater variations in relevance scores compared
 679 to the variations caused by biases alone. Conse-
 680 quently, the biases observed in our controlled ex-
 681 periments may have a less significant effect on the
 682 ranking of real documents, as the influence of con-
 683 text differences could be more dominant.

684 References

- 685 Adam Berger and John Lafferty. 1999. Information re-
 686 trieval as statistical translation. In *Proceedings of*
 687 *the 22nd annual international ACM SIGIR confer-*
 688 *ence on Research and development in information*
 689 *retrieval*, pages 222–229.
- 690 Leonid Boytsov and Zico Kolter. 2021. Exploring clas-
 691 sic and neural lexical translation models for infor-
 692 mation retrieval: Interpretability, effectiveness, and
 693 efficiency benefits. In *Advances in Information Re-*
 694 *trieval: 43rd European Conference on IR Research,*
 695 *ECIR 2021, Virtual Event, March 28–April 1, 2021,*
 696 *Proceedings, Part I* 43, pages 63–78. Springer.
- 697 Olcay Boz. 2002. Extracting decision trees from
 698 trained neural networks. In *Proceedings of the*
 699 *eighth ACM SIGKDD international conference on*
 700 *Knowledge discovery and data mining*, pages 456–
 701 461.
- 702 Tanya Chowdhury, Razieh Rahimi, and James Allan.
 703 2023. Rank-lime: local model-agnostic feature at-
 704 tribution for learning to rank. In *Proceedings of the*
 705 *2023 ACM SIGIR International Conference on The-*
 706 *ory of Information Retrieval*, pages 33–37.

- Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and
 Daniel Campos. 2021. [Overview of the trec 2020
 deep learning track](#).
- Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel
 Campos, and Ellen M Voorhees. 2020. Overview of
 the trec 2019 deep learning track.
- Mark Craven and Jude Shavlik. 1995. Extracting tree-
 structured representations of trained networks. *Ad-*
vances in neural information processing systems, 8.
- W Bruce Croft, Donald Metzler, and Trevor Strohman.
 2010. *Search engines: Information retrieval in*
practice, volume 520. Addison-Wesley Reading.
- Zhuyun Dai and Jamie Callan. 2019. Deeper text un-
 derstanding for ir with contextual neural language
 modeling. In *Proceedings of the 42nd International*
ACM SIGIR Conference on Research and Develop-
ment in Information Retrieval, pages 985–988.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and
 Kristina Toutanova. 2019. [BERT: Pre-training of
 deep bidirectional transformers for language under-](#)
[standing](#). In *Proceedings of the 2019 Conference*
of the North American Chapter of the Association
for Computational Linguistics: Human Language
Technologies, Volume 1 (Long and Short Papers),
 pages 4171–4186, Minneapolis, Minnesota. Associ-
 ation for Computational Linguistics.
- Thibault Formal, Carlos Lassance, Benjamin Pi-
 wowski, and Stéphane Clinchant. 2021a. Splade
 v2: Sparse lexical and expansion model for infor-
 mation retrieval. *arXiv preprint arXiv:2109.10086*.
- Thibault Formal, Benjamin Piwowski, and Stéphane
 Clinchant. 2021b. Splade: Sparse lexical and ex-
 pansion model for first stage ranking. In *Proceed-*
ings of the 44th International ACM SIGIR Confer-
ence on Research and Development in Information
Retrieval, pages 2288–2292.
- Luyu Gao, Zhuyun Dai, and Jamie Callan. 2021. Coil:
 Revisit exact lexical match in information retrieval
 with contextualized inverted list. In *Conference of*
the North American Chapter of the Association for
Computational Linguistics: Human Language Tech-
nologies.
- Riccardo Guidotti, Anna Monreale, Salvatore Rug-
 gier, Franco Turini, Fosca Giannotti, and Dino Pe-
 dreschi. 2018. A survey of methods for explaining
 black box models. *ACM computing surveys (CSUR)*,
 51(5):1–42.
- Xiaochuang Han, Byron C Wallace, and Yulia
 Tsvetkov. 2020. Explaining black box predictions
 and unveiling data artifacts through influence func-
 tions. In *Proceedings of the 58th Annual Meeting of*
the Association for Computational Linguistics. As-
 sociation for Computational Linguistics.

760	Sebastian Hofstätter, Sheng-Chieh Lin, Jheng-Hong	Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee.	816
761	Yang, Jimmy Lin, and Allan Hanbury. 2021. Effi-	2020. From local explanations to global understand-	817
762	ciently teaching an effective dense retriever with bal-	ing with explainable ai for trees. <i>Nature machine</i>	818
763	anced topic aware sampling. In <i>Proceedings of the</i>	<i>intelligence</i> , 2(1):56–67.	819
764	<i>44th International ACM SIGIR Conference on Re-</i>		
765	<i>search and Development in Information Retrieval</i> ,	Lijun Lyu and Avishek Anand. 2023. Listwise expla-	820
766	pages 113–122.	nations for ranking models using multiple explain-	821
		ers. In <i>European Conference on Information Re-</i>	822
767	Gautier Izacard, Mathilde Caron, Lucas Hosseini, Se-	<i>trieval</i> , pages 653–668. Springer.	823
768	bastian Riedel, Piotr Bojanowski, Armand Joulin,		
769	and Edouard Grave. 2021. Unsupervised dense in-	Sean MacAvaney, Andrew Yates, Arman Cohan, and	824
770	formation retrieval with contrastive learning. <i>arXiv</i>	Nazli Goharian. 2019. Cedr: Contextualized em-	825
771	<i>preprint arXiv:2112.09118</i> .	beddings for document ranking. In <i>Proceedings of</i>	826
		<i>the 42nd international ACM SIGIR conference on</i>	827
772	Joseph D Janizek, Pascal Sturmfels, and Su-In Lee.	<i>research and development in information retrieval</i> ,	828
773	2021. Explaining explanations: Axiomatic feature	pages 1101–1104.	829
774	interactions for deep networks. <i>Journal of Machine</i>		
775	<i>Learning Research</i> , 22(104):1–54.	Chandler May, Alex Wang, Shikha Bordia, Samuel	830
		Bowman, and Rachel Rudinger. 2019. On measur-	831
776	Omar Khattab and Matei Zaharia. 2020. Colbert: Ef-	ing social biases in sentence encoders. In <i>Proceed-</i>	832
777	ficient and effective passage search via contextual-	<i>ings of the 2019 Conference of the North American</i>	833
778	ized late interaction over bert. In <i>Proceedings of the</i>	<i>Chapter of the Association for Computational Lin-</i>	834
779	<i>43rd International ACM SIGIR Conference on Re-</i>	<i>guistics: Human Language Technologies, Volume 1</i>	835
780	<i>search and Development in Information Retrieval</i> ,	(<i>Long and Short Papers</i>), pages 622–628.	836
781	pages 39–48.		
		Shahrazad Naseri, Jeffrey Dalton, Andrew Yates, and	837
782	Youngwoo Kim, Myungha Jang, and James Allan.	James Allan. 2021. Ceqe: Contextualized embed-	838
783	2020. Explaining text matching on neural natural	dings for query expansion. In <i>Advances in Informa-</i>	839
784	language inference. <i>ACM Transactions on Informa-</i>	<i>tion Retrieval: 43rd European Conference on IR Re-</i>	840
785	<i>tion Systems (TOIS)</i> , 38(4):1–23.	<i>search, ECIR 2021, Virtual Event, March 28–April</i>	841
		<i>1, 2021, Proceedings, Part I 43</i> , pages 467–482.	842
786	Youngwoo Kim, Razieh Rahimi, and James Allan.	Springer.	843
787	2022. Alignment rationale for query-document rel-		
788	evance. In <i>Proceedings of the 45th International</i>	Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao,	844
789	<i>ACM SIGIR Conference on Research and Develop-</i>	Saurabh Tiwary, Rangan Majumder, and Li Deng.	845
790	<i>ment in Information Retrieval</i> , pages 2489–2494.	2016. Ms marco: A human generated machine read-	846
		ing comprehension dataset . In <i>CoCo@NIPS</i> .	847
791	Robert Krovetz. 1993. Viewing morphology as an in-		
792	ference process. In <i>Proceedings of the 16th an-</i>	Rodrigo Nogueira, Jimmy Lin, and AI Epistemic.	848
793	<i>annual international ACM SIGIR conference on Re-</i>	2019. From doc2query to docttttquery. <i>Online</i>	849
794	<i>search and development in information retrieval</i> ,	<i>preprint</i> , 6:2.	850
795	pages 191–202.		
		Saran Pandian, Debasis Ganguly, and Sean MacA-	851
796	Jimmy Lin. 2021. Github/castorini/anserini - clean up	vaney. 2024. Evaluating the explainability of neu-	852
797	garbage characters in ms marco dataset .	ral rankers. In <i>European Conference on Informa-</i>	853
		<i>tion Retrieval</i> , pages 369–383. Springer.	854
798	Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-		
799	Hong Yang, Ronak Pradeep, and Rodrigo Nogueira.	Jay M Ponte and W Bruce Croft. 1998. A language	855
800	2021. Pyserini: A Python toolkit for reproducible	modeling approach to information retrieval. In <i>Pro-</i>	856
801	information retrieval research with sparse and dense	<i>ceedings of the 21st annual international ACM SI-</i>	857
802	representations. In <i>Proceedings of the 44th Annual</i>	<i>GIR conference on Research and development in in-</i>	858
803	<i>International ACM SIGIR Conference on Research</i>	<i>formation retrieval</i> , pages 275–281.	859
804	<i>and Development in Information Retrieval (SIGIR</i>		
805	<i>2021)</i> , pages 2356–2362.	Martin F Porter. 1980. An algorithm for suffix strip-	860
		ping. <i>Program</i> , 14(3):130–137.	861
806	Michael Lloides, Debasis Ganguly, Sumit Bhatia, and		
807	Chirag Agarwal. 2023. Explain like i am bm25: In-	Dheeraj Rajagopal, Vidhisha Balachandran, Eduard H	862
808	terpreting a dense model’s ranked-list with a sparse	Hovy, and Yulia Tsvetkov. 2021. Selfexplain: A	863
809	approximation . In <i>Proceedings of the 46th Inter-</i>	self-explaining architecture for neural text classi-	864
810	<i>national ACM SIGIR Conference on Research and</i>	fiers. In <i>Proceedings of the 2021 Conference on</i>	865
811	<i>Development in Information Retrieval, SIGIR ’23</i> ,	<i>Empirical Methods in Natural Language Process-</i>	866
812	page 1976–1980, New York, NY, USA. Association	<i>ing</i> , pages 836–850.	867
813	for Computing Machinery.		
		Marco Tulio Ribeiro, Sameer Singh, and Carlos	868
814	Scott M Lundberg, Gabriel Erion, Hugh Chen, Alex	Guestrin. 2016. Why should i trust you?: Explain-	869
815	DeGrave, Jordan M Prutkin, Bala Nair, Ronit Katz,	ing the predictions of any classifier. In <i>Proceedings</i>	870

of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pages 1135–1144. ACM.

Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. [Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models](#). In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1. Curran.

Manisha Verma and Debasis Ganguly. 2019. Lirme: Locally interpretable ranking model explanation. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR’19, page 1281–1284.

A Unexpected behaviors

In subsection 4.3, we discovered three unexpected behaviors from the relevance thesaurus. This part describes the experiments that support the existence of the behaviors found in the state-of-the-art relevance models. The three behaviors investigated are (1) Postfix a, where the models treat “a” at the end of a word as a quotation mark or apostrophe due to encoding errors in the MS MARCO dataset; (2) Car-brand bias, where the models assign higher scores to certain car brand names over others; and (3) When-year bias, where the models exhibit temporal bias in assigning relevance scores to different years for queries containing the term “when”.

A.1 Car-brand bias

Brand name	Scores	Brand name	Scores
Volkswagen	0.429	Buick	0.308
Ferrari	0.410	Cadillac	0.303
Porsche	0.405	Renault	0.300
Fiat	0.394	Honda	0.279
Chrysler	0.390	Audi	0.269
Ford	0.389	Peugeot	0.269
Mercedes	0.377	Pontiac	0.259
Packard	0.366	Daimler	0.219
Oldsmobile	0.365	Mitsubishi	0.212
Toyota	0.350	Nissan	0.205
Jaguar	0.348	Chevrolet	0.202
Volvo	0.341	Lexus	0.180
Hyundai	0.332	Jeep	0.159
BMW	0.324	Mazda	0.094
Bentley	0.322		

Table 5: Scores for each of 29 brand names against the query term “car” based on our relevance thesaurus.

The relevance thesaurus reveals that the models associate the query term “car” with many brand names, such as “Ford” and “Honda”, but consistently assign higher scores to certain brand names over others (Table 5). To verify if this bias is present in the state-of-the-art relevance ranking models, we designed an experiment using the MS MARCO passage collection.

From the training split of the MS MARCO passage collection, we selected queries that include the term “car” but exclude any car brand names or content specific to particular brands. We then selected documents for each of the queries that satisfy the following criteria:

1. The document mentions only one brand name.

2. The document does not contain any brand-specific information when the brand name is removed.
3. The document is predicted as relevant by the cross-encoder model.

To filter the documents based on the second and third criteria, we employed keyword-based filtering using a list of car models, ChatGPT-based filtering to identify brand-specific information and manual annotations. For the keyword filtering, we built a list of car models and excluded the documents that contained any of the model names. For ChatGPT-based filtering, we masked the brand name mention of the document and prompted, “Does this document contain any brand-specific information?”, and if the answer was yes, the document was excluded. This process resulted in 382 query-document pairs, with 29 car brand names considered.

For each query-document pair, the brand name mentioned in the document was replaced by each of the 29 brand names in turn. All the resulting combinations were scored by neural ranking models, yielding a score array of 382×29 , where each row represents a query-document pair and each column represents a brand name. In other words, the element at position (i, j) in the array represents the score assigned by the neural ranking model to the i -th query-document pair when the brand name is replaced by the j -th brand name. The element (i, i) represents the original brand that appeared in the document.

To obtain a single score per brand name, we averaged the scores across the 382 query-document pairs. We then measured the correlation between these average scores and the scores from the relevance table derived from the cross-encoder model. If a neural model exhibits the bias suggested by the representation, we expect a corresponding bias in the modified documents.

Table 6 shows the correlation values (fidelity) obtained for each of the ranking models. The results demonstrate that the scores from the thesaurus correlate with scores from the neural ranking models, indicating that our relevance thesaurus can be used to identify possible biases of ranking models.

A.2 When-year

The models exhibit a temporal bias where different years have different scores for the query term

Ranking Model	Car - brand	When - year
Cross Encoder	0.282	0.746
Splade v2	0.413	0.224
TAS-B	0.367	0.484
Contriever	0.419	0.422
Contriever + M	0.200	0.665

Table 6: The fidelity of relevance thesaurus focused on two findings. The models predict scores on query-document pairs when a brand name or year mention is replaced with another.

“when”, with much lower scores assigned to the years around 2015 compared to other years. Most years (e.g., “2001”) have high relevance scores for the query term “when” in the relevance thesaurus, but the score sharply decreased around 2015, the year when the MS MARCO dataset was constructed (Figure 4).

To validate if this bias is in neural models, We measured the predicted scores from the neural ranking models with the query being “when did North Carolina join IFTA” and the document being “year North Carolina join IFTA”. Table 6 shows the correlation between the scores from the ranking model and the relevance thesaurus.

The results show that the neural ranking models exhibit a similar temporal bias to the relevance thesaurus, where documents mentioning years around 2016 are scored lower in relevance for queries containing the term “when”. This correlation confirms that our relevance thesaurus faithfully captures the biases underlying the neural models.

A.3 Postfix A

Many entries in the thesaurus consist of cases where the document term is the query term with an additional “a” or “â” at the end, such as (“car”, “vehicleâ”). This is due to encoding errors in the MS MARCO dataset, where the right quotation marks (’) were incorrectly decoded as “â”. When combined with the BERT tokenizer normalizing “â” to “a”, it could result in incorrect matching, such as considering the term “cud” (food in cows’ stomach) to be relevant to “CUDA” (parallel computing platform).

To test the postfix hypothesis, we selected 500 relevant query-document pairs that contained a common term. We then appended characters from “a” to “z” to the document occurrence of this common term. For each appended character, we mea-

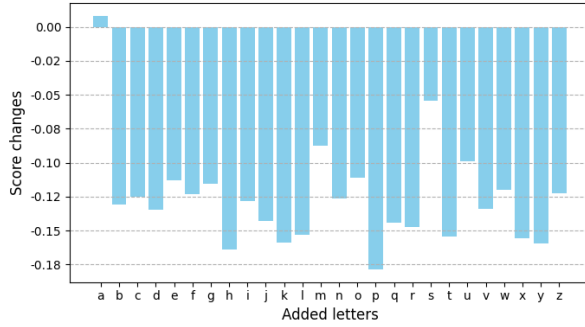


Figure 5: Postfix-a experiment results for the cross encoder. Modifying the matching document term by appending any alphabet results in a large score drop, except ‘a’.

sured the change in the relevance score assigned by the model.

The result on the cross encoder as illustrated in Figure 5 shows that while all other alphabet characters result in a large score drop when they are appended to the query term occurrence in the document, appending “a” actually results in a small increase of the relevance score. The difference between score changes of “a” and other cases are all statistically significant at $p < 0.01$. This supports the existence of many entries in the relevance thesaurus where the document term has the additional character “a” at the end of the terms.

The results, illustrated in Figure 5, show that appending “a” leads to a small increase in the relevance score, while all other characters result in a significant score drop ($p < 0.01$). This supports the existence of entries in the relevance thesaurus where the document term has an additional “a” at the end. Similar behaviors were observed on Splade and Contriever-MS MARCO, while Contriever and TAS-B do not exhibit such behavior (Table 7).

While the existence of encoding errors is known (Lin, 2021), there has been no systematic analysis of how these errors could affect the ranking models. This analysis shows that our thesaurus explanation can be effectively used to discover that the model is using features that may not generalize to other corpora.

B Experiments details

B.1 BM25T with Local to Global

As one of our baselines, we adapted the local explanation method proposed by Llordes et al. (2023) into a global explanation. Their approach provides local explanations for a given query and ranked documents by identifying matching document terms. The local explanation method can be represented as a function:

$$E : (q, d_1, d_2, \dots, d_n) \rightarrow (w_1, w_2, w_3, \dots)$$

, where q is a query, d_i is a ranked document, and w_i is a document term that is considered to match the query.

While it has been argued that local explanations can be converted into global explanations by accumulating them (Janizek et al., 2021), the local explanations for IR models often lack the necessary information to be used globally. Specifically, the document terms (w_i) are not attributed to specific query terms, which is a crucial requirement for building a term-level global thesaurus.

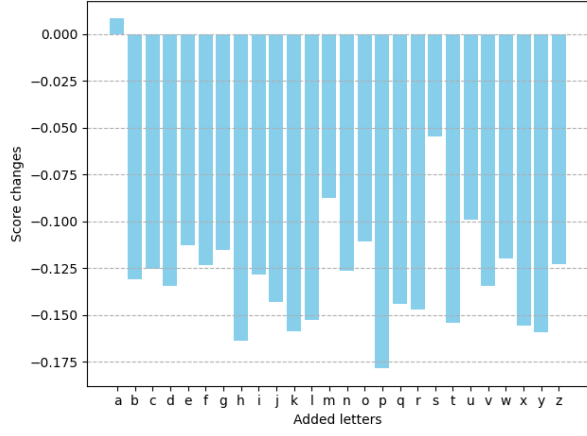
To build a global explanation in the form of a relevance thesaurus, we align each document term to one of the query terms using the following method. Using a cross-encoder ranker, we compute relevance scores between each query term and each document term, considering them as standalone queries and documents, respectively. The query term with the highest score is aligned to the corresponding document term.

We apply this alignment process to 400K training queries and their re-ranked candidate documents using the cross-encoder. Compared to PaRM, this method uses a similar number of queries but a larger number of documents per query. For each query, 1,000 candidate documents were re-ranked and the top 10 documents were used to select the document terms for explanation building.

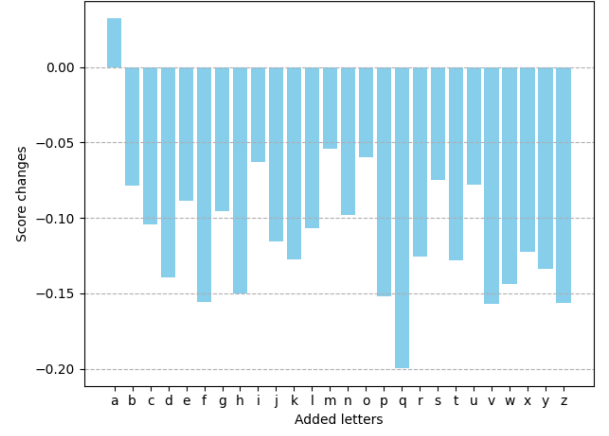
The score between a query term qt and a document term dt is calculated as the number of times qt is aligned to dt divided by the total number of occurrences of qt in the queries. We only included the document terms which appear more than once, to reduce the noise. Table 8 shows the result for the different variants for BM25T (L to G).

B.2 QLT

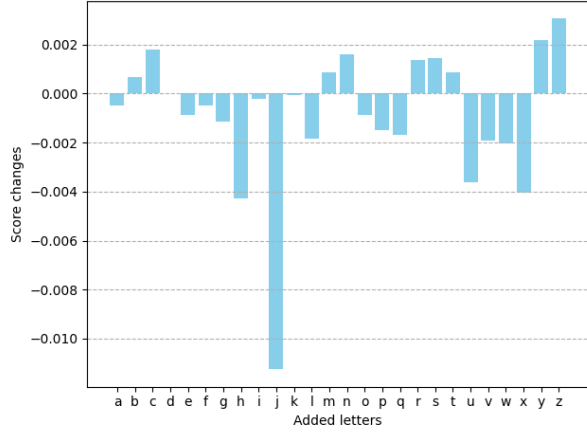
We implemented and evaluated the performance of QLT (Query Likelihood with Thesaurus) to test



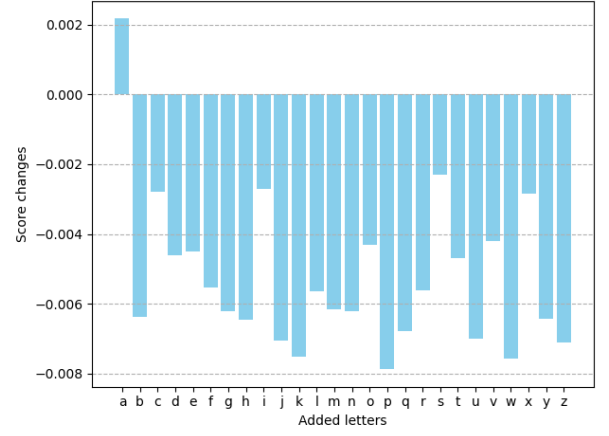
Cross-encoder



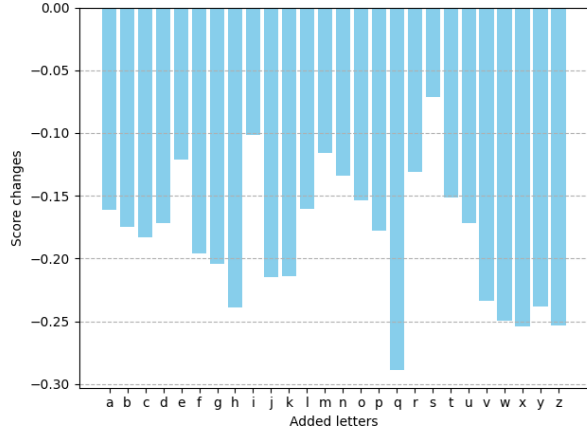
Splade v2



Contriever



Contriever+MS MARCO



TAS-B

Table 7: Postfix-a experiments results. The listed scores are average of (score after change — score before change), and positive values indicate score increases and negative values indicate score decreases.

	TREC DL 19	TREC DL 20
BM25	0.516	0.503
BM25T		
PaRM	0.550	0.546
L to G	0.504	0.501
+ Min frequency filter	0.518	0.501
+ Uniform attribution	0.475	0.493

Table 8: The ranking effectiveness of BM25T (L to G) with different configurations

how the relevance thesaurus can be used for different models that the thesaurus was not optimized for. One key difference between QLT and BM25T is that QLT computes the score as the sum of the relevance scores of the document terms, while BM25T computes the query term’s score based on the most relevant document term.

In the original query likelihood model, the relevance score for query q and document d is computed as:

$$p(\mathbf{q}|\mathbf{d}) = \prod_i p(q_i|\mathbf{d}), \quad (8)$$

where $p(q_i|\mathbf{d})$ is the probability of the query term q_i in the document d , calculated as the frequency of q_i in d divided by the length of d .

The translation language model for information retrieval (Berger and Lafferty, 1999) considers that any document term w can be “translated” into the query term q_i with the translation probability $t(q_i|w)$. The term probability is then computed as:

$$p(q_i|\mathbf{d}) = \sum_w t(q_i|w)p(w|\mathbf{d}), \quad (9)$$

where $t(q_j|w)$ is the translation probability, and $p(w|\mathbf{d})$ is the probability of the word w in the document d , again computed as the frequency of q_i in d divided by the length of d . In QLT, we adopt the translation language model while using our relevance thesaurus to compute the translation probability.

B.3 BM25 and Query Likelihood (QL) configuration

For the hyper-parameters of BM25, we used the default values ($k_1 = 0.9$ and $b = 0.4$) as in Pyserini (Lin et al., 2021). We used the analyzer configurations as in the Pyserini implementation, which includes normalization, tokenization, stemming, and stopwords removal.

Our query likelihood implementation uses the same tokenizer as in BM25. We used Dirichlet

smoothing (Croft et al., 2010), and tuned the parameter μ on another validation set, as the default values were ineffective for short passages.

C Full experiments results

C.1 Different fidelity metrics

Many works have used different metrics for the fidelity evaluation of ranked lists in IR tasks, such as the overlap of **top-k** in the ranked list (Llordes et al., 2023), agreement rates of **pairwise** preference agreement in the ranked list (Lyu and Anand, 2023), or **Kendall** rank correlations (Pandian et al., 2024).

We chose the Pearson correlation as the main metric because it considers the magnitude of score differences rather than just the rankings. This is particularly important in IR tasks, where the emphasis is on differentiating a few highly relevant documents from the many non-relevant ones.

While IR tasks are indeed ranking tasks, they prioritize top-ranked documents, which are more likely to be relevant. As a result, ranking correlations like the Kendall rank correlation may be less appropriate, as they are more affected by the ranking of non-relevant documents, which outnumber relevant ones. We also consider the overlap rates of top-k to be less desirable for two reasons: first, the number of relevant documents is unknown; second, it does not account for score differences among the top-k items, which are crucial for reliable IR metrics like NDCG. Nevertheless, we have included results for other fidelity metrics (Table 9), which consistently confirm the improvements of BM25T (PaRM) over BM25.

C.2 Fidelity on BEIR

In subsection 4.2, we include only the ranking effectiveness of BM25T on the BEIR dataset, and not the fidelity (correlation). Table 10 shows the correlation of the scores when BM25 or BM25T is considered as an explanation and compared to scores from the cross-encoder model. In most datasets, the correlations increased, with the only exception of the Quora dataset.

The improvements in ranking performance are consistent with the increased correlations observed in most datasets.

D PaRM implementation details

In the first phase, PaRM calculates a relevance score for the given query-document by generating

Model	Pearson r		Kendall τ		Pairwise		Top-k overlap	
	BM25	BM25T	BM25	BM25T	BM25	BM25T	BM25	BM25T
Cross Encoder	0.484	0.580	0.260	0.341	0.632	0.672	0.293	0.334
Splade v2	0.490	0.583	0.268	0.346	0.636	0.674	0.304	0.345
TAS-B	0.421	0.513	0.228	0.304	0.616	0.653	0.256	0.293
Contriever	0.417	0.454	0.230	0.259	0.617	0.631	0.273	0.289
Contriever + M	0.411	0.495	0.225	0.294	0.615	0.648	0.264	0.303

Table 9: Fidelity of BM25 and BM25T when measured by different metrics.

Dataset	BM25	BM25T
HotpotQA	0.535	0.647
DBPedia	0.477	0.612
NQ	0.474	0.658
Touché-2020	0.403	0.689
SCIDOCS	0.598	0.663
TREC-COVID	0.276	0.705
FiQA-2018	0.481	0.514
Quora	0.659	0.640
ArguAna	0.656	0.722
SciFact	0.634	0.677
NFCorpus	0.584	0.626
ViHealthQA	0.314	0.410

Table 10: Fidelity (Pearson correlation) of the BM25 and BM25T as explanation to the cross encoder ranking model.

scores for two inputs, (q_1, d_1) and (q_2, d_2) , which are built from the given query q and document d . Each of the two inputs is then scored through PaRM and these two scores are summed as the relevance score for the query-document pair. Using the relevance label for the query and document, PaRM is trained end-to-end to predict relevance for partial sequences of the query and document without fine-grained labels.

We build q_1 by extracting a continuous span from q and build q_2 with the remaining tokens, leaving a [MASK] token where q_1 was extracted. Given q_1, q_2 , and d , we build d_1 and d_2 by masking some tokens of the document d , while keeping tokens that are likely to be relevant to the corresponding q_i . Both d_1 and d_2 can be composed of many non-continuous spans.

D.1 Building partial segments for the first phase

The tokens to be deleted are selected so that the deleted tokens in d_i are less likely to be important for the corresponding query partition q_i . To

estimate which tokens of the document are less likely to be important for a query partition, we use the attention scores from a canonical cross-encoder model which takes the concatenation of whole query q and document d as an input.

The scoring is done in the following steps.

1. We collect normalized attention probabilities from all the layers and heads of the Transformer network. As a result, we get a four dimension tensor of $W \in \mathbb{R}^{L \times L \times M \times H}$, where L is the sequence length, M is the number of layers, and H is the number of attention heads in each layer. W_{ijkl} denote the attention probability for the i -th token to attend to the j -th token in the k -th attention head of the l -th layer.

2. We average W over the last two dimensions, which correspond to different layers and heads, and get a two-dimensional matrix A .

$$A_{ij} = \sum_l \sum_k W_{ijkl} \quad (10)$$

3. Let $|q|$ be the number of tokens in the query and $|d|$ to be the number of tokens in the document. When a [CLS] token and [SEP] tokens are combined with the query and document tokens, the query tokens are located from the second token to $(|q| + 1)$ -th token, and the document tokens are located from $(|q| + 3)$ -th token to $(|q| + |d| + 2)$ -th.

Then, $A_{2:|q|+1, |q|+3:|q|+|d|+2}$ indicates the averaged probability that query tokens to attend to document tokens, and $A_{|q|+3:|q|+|d|+2, 2:|q|+1}$ indicates averaged probability that document tokens attend to query tokens.

4. By transposing the latter matrix and adding it to the first, we obtain S . In this resultant

matrix, S_{ij} indicates the degree of attention between the i -th token of the query and j -th token of the document.

To avoid splitting a word into subwords, the token selection was performed at the word-token level instead of the subword-token. The subword-token-level scores are converted by taking maximum scores.

We select the tokens with the lowest attention probability for the corresponding query partition as ones to be deleted.

The number of tokens to be deleted from the document is randomly sampled so that it can have variable inputs starting from a single token to nearly a full sequence. The number of deleted tokens m is sampled from a normal distribution with the mean and standard deviation being half of the document length $|d|$. The sampled number is capped at a minimum of 1 and a maximum of $|d| - 1$.

D.2 Second Phase

In the second phase, PaRM is trained on word pairs, where the query term does not appear in the document. Here are a few clarifications about the details.

To select term-pair candidates, we pre-computed scores for the term pairs by limiting them to the frequent terms. Similar to the relevance thesaurus itself, the top 10K frequent terms were considered as query terms, and the top 100K terms as document terms. The terms without scores are not selected for training.

The terms that are fed to PaRM are stemmed in the same way they are used for BM25T. However, stopwords are NOT excluded for this step, while BM25T excludes them.

D.3 Training configurations

Both phases use the query-documents triplets provided with MS MARCO passage ranking dataset (Nguyen et al., 2016), which makes about 400K training instances.

For the first stage training of PaRM, we apply early stopping based on the loss validation set. We used the batch size of 16 and learning rate of $2e-5$, which were not tuned.

For the second stage of training of PaRM, we tuned the learning rate and batch sizes based on its loss on holdout split and BM25T augmented

performance (MRR) in a validation set that is separately sampled from MS MARCO dev. The reported model used a batch size of 256 and a learning rate of $1e-5$.

D.4 Computational cost

We used four NVidia GTX 1080 ti GPUs for training. Each of the first and second stages of training took less than 10 hours. The inference for relevance thesaurus construction was run on a large (100) number of GPUs, which have similar performance to GTX 1080 ti and took about 700 GPU hours.

E Responsibility statement

E.1 Artifact - MS MARCO Dataset

The MS MARCO dataset (Nguyen et al., 2016), used as the artifact in this paper, has been carefully curated and anonymized by its creators to protect user privacy and prevent the inclusion of personally identifying information or offensive content. The dataset consists of anonymized search queries and corresponding relevant passages from web pages, processed to remove any personal information.

The MS MARCO dataset is a large-scale information retrieval dataset covering a wide range of domains and topics in the English language. It includes real-world search queries from Bing and corresponding relevant passages from web pages. The dataset is divided into training, development, and testing sets³, each containing a substantial number of query-passage pairs. While demographic information is not explicitly provided due to privacy concerns, the dataset is considered representative of diverse information needs and user intents in web search scenarios.

F Acknowledgement

F.1 AI Assistance

We acknowledge the use of AI assistants, Claude by Anthropic⁴ and GPT-4 by OpenAI⁵, in the writing process of this paper. These AI assistants provided support in drafting and refining the contents of the paper. However, all final decisions regarding the content, structure, and claims were made by the human authors, who carefully reviewed and edited the generated content.

³<https://microsoft.github.io/msmarco/Datasets>

⁴<https://www.anthropic.com/claude>

⁵<https://chat.openai.com/>